CS3520 · COMPUTER ORGANISATION AND ARCHITECTURE · NATIONAL UNIVERSITY OF LESOTHO

LIMO Simulator

Design Document — Assembler and Five-Stage Pipeline

Team 7 — Realeboha Makhaleme · Bahlakoana Mosola · Kobo Ts'epo Ramosoeu · Katleho Senqhe · Khauhelo Moleboheng Mosanteli · Tumelo Khosola · Mweli Pama

Repository: github.com/realebohamakhaleme68-arch/LIMO-SESOTHO-LANGUAGE-PROCESSOR

Document: preliminary submission · October 2026

> This document describes the design of the LIMO simulator and distinguishes the intended design from what currently runs. Where a capability is implemented and tested, it is marked as such; where it is still being built, it is marked as in progress. No unfinished functionality is described as complete.

1. Project overview, requirements, objectives and scope

LIMO is a 32-bit load–store processor derived from RISC-V (RV32I) whose assembly language is Sesotho. The project has two deliverables: the LIMO instruction set architecture (Milestone 1, submitted separately as the ISA specification), and an in-browser simulator that brings the architecture to life. This document covers the simulator.

The simulator is a static web application that runs entirely in the browser — no server, login or paid service — so that a first-year student can type a LIMO program, assemble it, and step it through a five-stage pipeline one clock cycle at a time, seeing the datapath values, the control signals, and how each hazard is detected and resolved.

Objectives

- Assemble LIMO source (Sesotho mnemonics) into correct machine code, with clear errors.
- Execute programs on a faithful five-stage pipeline with forwarding, stalls and flushes.
- Make the datapath, control signals, hazards and statistics visible and explainable.
- Reproduce the expected results of the three ISA sample programs exactly.

Scope

In scope: the full 15-instruction LIMO set (R, I, S, B and J formats), a 16-register file, separate instruction and data memories, and the single-issue in-order five-stage pipeline. Out of scope (by ISA design): multiply/divide, floating point, control and status registers, exceptions and a flags register.

2. System architecture and application design

The application separates a pure execution engine from the browser interface. The engine is the single source of truth; the interface only renders engine state. The same engine is used by the automated tests (under Node) and by the browser, so there is no second, divergent "display-only" model.

Layer	Responsibility	Status
`core` (ISA)	Register aliases, instruction table, encode and decode.	Implemented, tested
`assembler`	Two-pass assembly, label resolution, validation, diagnostics.	Implemented, tested
`pipeline`	Five-stage execution, latches, forwarding, hazards, faults, statistics, snapshots.	Implemented, tested
`ui`	Editor, machine-code view, registers, memory, datapath, control, chart, narration.	In progress

The engine is written in TypeScript and compiled to plain JavaScript that the page loads directly; the compiled output is kept in the repository so the static site runs on GitHub Pages with no build step for the viewer. The interface (`index.html` and `style.css`) defines the editor, machine-code table, register and memory views, pipeline stage panels, control-signal table, pipeline chart, hazard log and the execution controls (Step, Step Back, Run, Pause, Run to End, Reset, a Forwarding switch and a Freshman-mode switch).

3. LIMO instruction set, registers, formats and encoding

Register file

Sixteen 32-bit registers, r0–r15. Three are special: `noto` (r0) is hard-wired to zero; `khutla` (r1) holds the return address; `lesupa` (r2) is the stack pointer. The remaining registers r3–r15 are general purpose and are named `lb3`…`lb15`. The program counter `sebali` is not a register-file operand. Register fields are 5 bits for RV32I bit-compatibility, with bit 4 unused for r0–r15.

Instruction formats

Fixed 32-bit instructions in five RV32I-derived formats:

Format	31–25	24–20	19–15	14–12	11–7	6–0
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
J	imm[20|10:1|11|19:12]				rd	opcode

Instruction set

Sesotho	RV32I	Fmt	opcode	funct3	funct7	Operation
kopanya	add	R	0110011	000	0000000	rd = rs1 + rs2
tlosa	sub	R	0110011	000	0100000	rd = rs1 − rs2
checha-menahane	sll	R	0110011	001	0000000	rd = rs1 << rs2
papisonyane	slt	R	0110011	010	0000000	rd = (rs1 < rs2) ? 1 : 0
kapafeela	xor	R	0110011	100	0000000	rd = rs1 ^ rs2
checha-arola	srl	R	0110011	101	0000000	rd = rs1 >> rs2 (logical)
kapa	or	R	0110011	110	0000000	rd = rs1 | rs2
le	and	R	0110011	111	0000000	rd = rs1 & rs2
kopanyapalo	addi	I	0010011	000	—	rd = rs1 + imm
lata	lw	I	0000011	010	—	rd = mem[rs1 + imm]
qhoma-aterese	jalr	I	1100111	000	—	rd = PC+4; PC = (rs1+imm) & ~1
boloka	sw	S	0100011	010	—	mem[rs1 + imm] = rs2
tlola-lekana	beq	B	1100011	000	—	if rs1 = rs2: PC += offset
tlola-fapana	bne	B	1100011	001	—	if rs1 ≠ rs2: PC += offset
qhoma	jal	J	1101111	—	—	rd = PC+4; PC += offset

Immediate ranges: I/S signed [−2048, 2047]; B signed even [−4096, 4094]; J signed even [−1 048 576, 1 048 574]. Arithmetic is 32-bit two's complement. Data memory is 4 KiB, little-endian, word-aligned.

Worked encodings (verified by the assembler's tests)

Assembly	Machine word
`kopanya r7, r5, r6`	`0x006283B3`
`kopanyapalo r5, r0, 10`	`0x00A00293`
`lata r8, 0(r9)`	`0x0004A403`
`boloka r6, 8(r2)`	`0x00612423`
`tlola-lekana r5, r6, 8`	`0x00628463`
`qhoma r1, 16`	`0x010000EF`

4. The five-stage pipeline and pipeline registers

LIMO uses a single-issue, in-order pipeline with five stages and four pipeline registers between them.

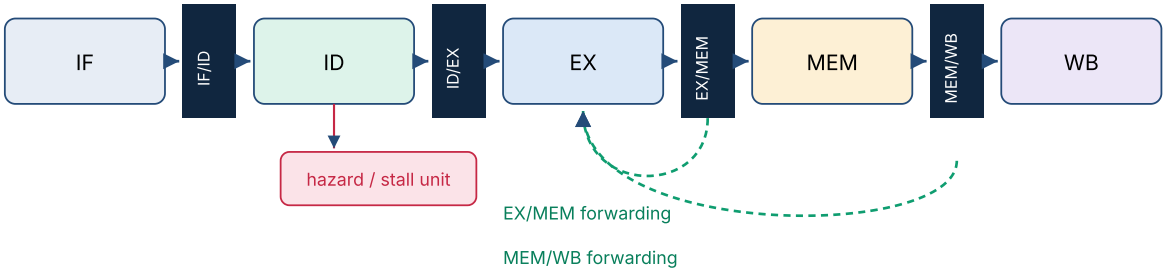


Figure 1 — the five stages, the four pipeline registers, the forwarding paths into EX, and the hazard unit that inserts stalls in ID.

Each pipeline register carries the in-flight instruction together with the values computed so far. The engine advances exactly one clock cycle per Step: write-back happens in the first half of the cycle (so a value written by the instruction in WB is visible to an instruction reading registers in the same cycle), then MEM, EX, ID and IF are resolved so that each pipeline register is written only after all of its inputs for the cycle are captured.

Stage	What it does in LIMO
IF	Fetch the instruction at PC from instruction memory; compute PC + 4.
ID	Decode; read source registers; generate control signals; detect hazards.
EX	ALU operation or effective-address calculation; resolve branches and jumps.
MEM	Word load or store on data memory; reject invalid or misaligned addresses.
WB	Write the result back to the register file (writes to r0 are discarded).

5. Control signals, hazards, forwarding, stalls and flushes

Control signals

Control is decoded in ID and carried down the pipeline registers. The signals are RegWrite, ALUSrc, MemRead, MemWrite, MemtoReg, Branch, Jump, PCSrc and the ALU operation.

Instruction class	RegWrite	ALUSrc	MemRead	MemWrite	MemtoReg	Branch	Jump
R-type	1	0	0	0	0	0	0
kopanyapalo (addi)	1	1	0	0	0	0	0
lata (lw)	1	1	1	0	1	0	0
boloka (sw)	0	1	0	1	0	0	0
branches	0	0	0	0	0	1	0
qhoma / qhoma-aterese	1	—	0	0	0	0	1

Data hazards and forwarding

A read-after-write dependency between a producing instruction and a close consumer is resolved by forwarding: when forwarding is on, the ALU takes its operand from the EX/MEM register (an ALU result one stage ahead) or the MEM/WB register (a result two stages ahead) rather than the stale register file. Store data is forwarded the same way, so a value produced just before a store reaches memory correctly.

The load-use stall

A loaded value is only available at the end of MEM, so an instruction that uses the result of an immediately preceding `lata` cannot be served by forwarding alone. The hazard unit detects this (a load in EX whose destination is a source of the instruction in ID) and inserts exactly one stall: it holds IF and ID and sends a bubble into EX. The next cycle the loaded value forwards from MEM/WB.

Control hazards

Branches and jumps are resolved in EX with always-not-taken prediction, reusing the ALU; this needs no extra comparator. When a transfer is taken, the two younger instructions already fetched (in ID and IF) are flushed — two bubbles per taken transfer — and fetch resumes from the target. A not-taken branch costs nothing. Flushed instructions never commit a register or memory effect.

Forwarding on versus off

A Forwarding switch exposes the cost of hazards. With forwarding off, the hazard unit instead stalls the consumer in ID until the producer's value is in the register file (writes occur in the first half of a cycle, reads in the second). This changes only the cycle count, never the architectural result.

6. Assembler design, validation and error reporting

The assembler is two-pass. The first pass strips comments, records label addresses (instructions are four bytes, addresses increase by four) and collects each instruction. The second pass parses operands by format, resolves labels to PC-relative offsets, checks ranges, and encodes each instruction to its 32-bit word.

- **Lexing:** labels end with a colon; # starts a comment; immediates are decimal or `0x` hex; matching is case-insensitive; an apostrophe inside a word is stripped before lookup (the ISA's ts'/ch' rule), while an apostrophe inside a number is an error.
- **Registers:** each register is accepted by its Sesotho name, its `lbN`/`rN`/`xN` form and documented aliases; r0 accepts both `noto` and `lefela`.
- **Validation:** unknown mnemonic or register, wrong operand count, empty operand, immediate out of range, odd or out-of-range branch/jump offset, undefined label and duplicate label are all reported, each with its 1-based source line number.
- **Targets:** a label resolves to (label address − instruction address); a numeric branch/jump operand is a signed PC-relative byte offset.

Lookup tables are built without a prototype chain so that names such as `constructor` cannot be mistaken for a valid register, instruction or label. Effective-address alignment and bounds depend on a register's run-time value and so are enforced at run time by the pipeline, which raises a fault rather than wrapping or corrupting memory. A program that fails to assemble cannot be run.

7. Test strategy, example programs and implementation status

Correctness is checked by an automated suite runnable with a single command (`npm test`): assembler tests (encodings, aliases, labels, immediate boundaries, error cases) and pipeline tests (stage movement, forwarding on and off, the load-use stall, branch flushes, register-zero invariance, runtime faults, Step Back, and the three sample programs end to end).

Acceptance programs

The three sample programs from the ISA document are the acceptance tests. Their expected registers, memory and cycle accounting come from the ISA and were not altered to fit the implementation; the implementation reproduces them.

Program	Committed	Stalls	Flushes	Cycles	Key final state
P1 — arithmetic / logic	11	0	0	15	lb5=22, lb10=48, lb13=1
P2 — memory + load-use	8	1	0	13	lb7=12; mem[0]=5, mem[4]=7, mem[8]=12
P3 — loop, branch, call	15	0	6	25	lb4=6, lb5=99, khutla=32

Cycle accounting follows cycles = committed + 4 + stalls + flushes. P3 executes 15 dynamic instructions from 10 source lines because the loop body runs three times; the two taken loop branches and the one taken call contribute six flush bubbles.

> **Implementation status (honest).** Implemented and passing the automated suite: the ISA core, the two-pass assembler, and the five-stage pipeline engine including forwarding, the load-use stall, branch/jump flushing, runtime faults, statistics and Step Back. In progress: wiring the existing browser interface to this engine (the datapath, control, chart and narration panels) and the remaining example and usability-test deliverables. Nothing described here as working is unverified — every reported figure is produced by running the tests.